System Verilog for Verification

Aman Arora

Outline

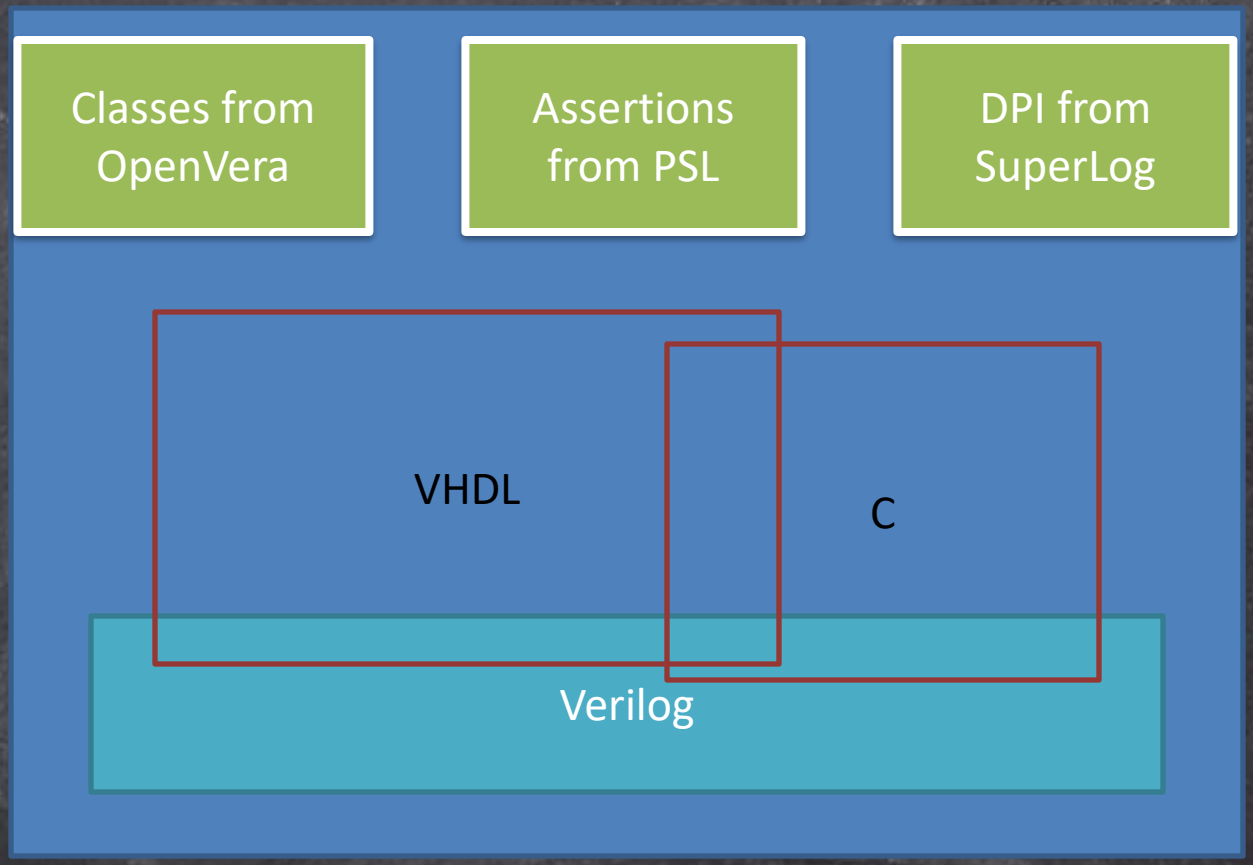
- ✓Introduction
- ✓Running example
- ✓Software inspired features
  - ✓Data types
  - ✓OOP
  - ✓Threads
- ✓Connection between the HW and the SW worlds
- ✓Hardware inspired features
  - ✓Stimulus
  - ✓Coverage
  - ✓Checking
- ✓Putting it all together - A testbench
- ✓Final thoughts

Introduction

System Verilog

- ✓System Verilog is an HDVL
  - ✓Hardware description language meets...
  - ✓High level parallel programming language
- ✓Verilog 2009 was the last version of Verilog
- ✓System Verilog is an IEEE Standard (1800-2012)
- ✓Most new features are not-synthesizable, they are meant for verification
- ✓Here is the LRM: SystemVerilog LRM

System Verilog



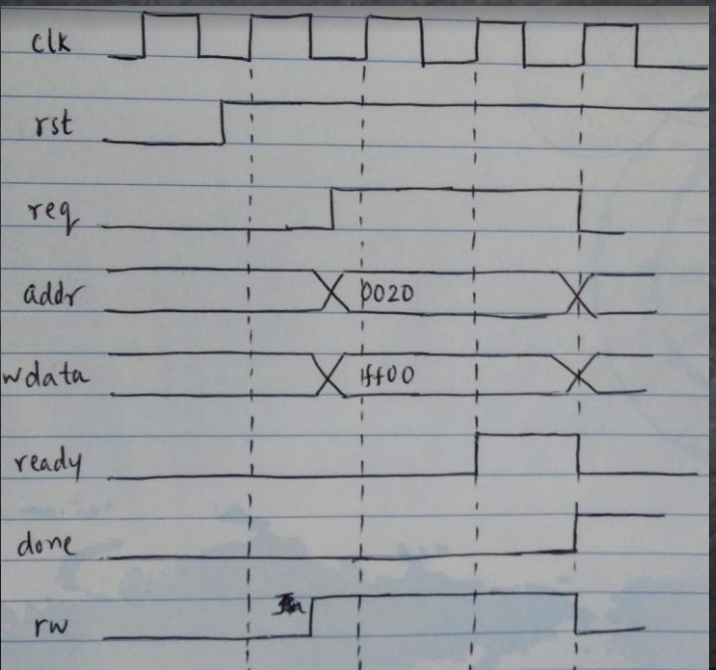
Source: doulos.com

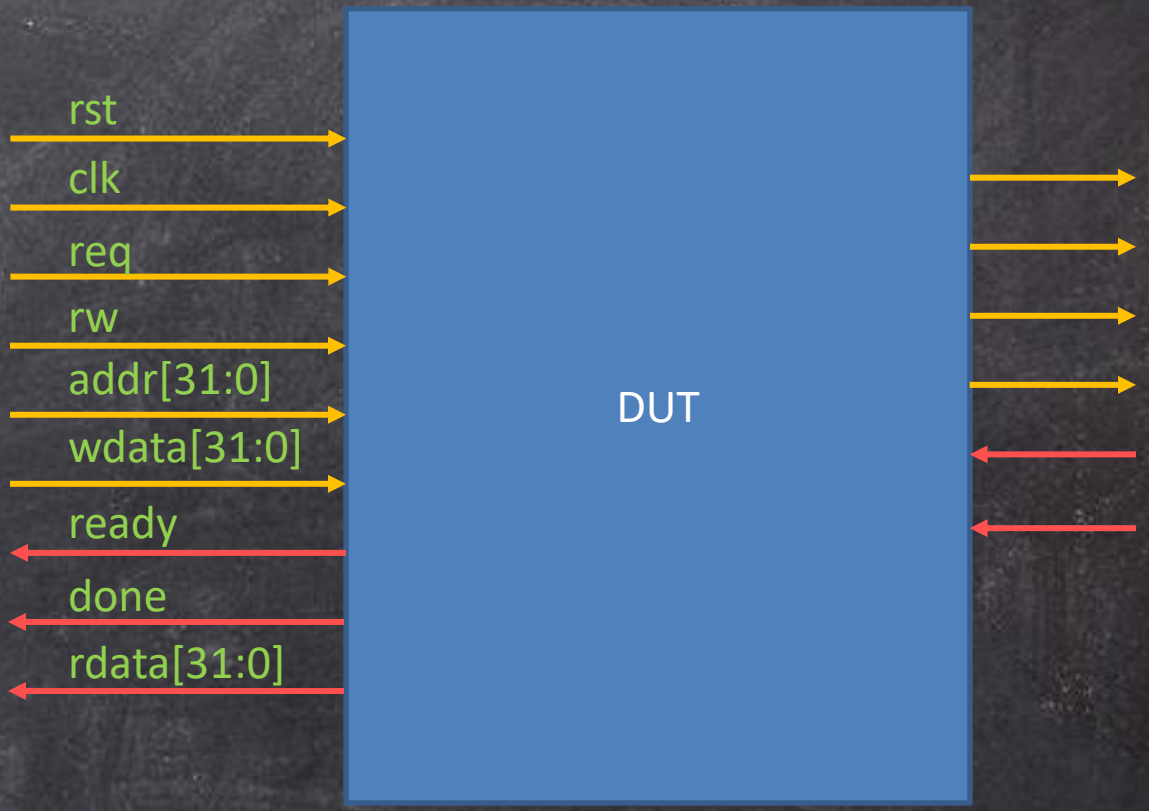
Aside: UVM

- Universal Verification Methodology
  - A methodology and a library written in System Verilog
  - Started by Acellera, but now an IEEE Standard
  - Supported by all EDA vendors
- VMM, OVM, eRM…… finally UVM
- Standardization and reuse
  - Verification IP

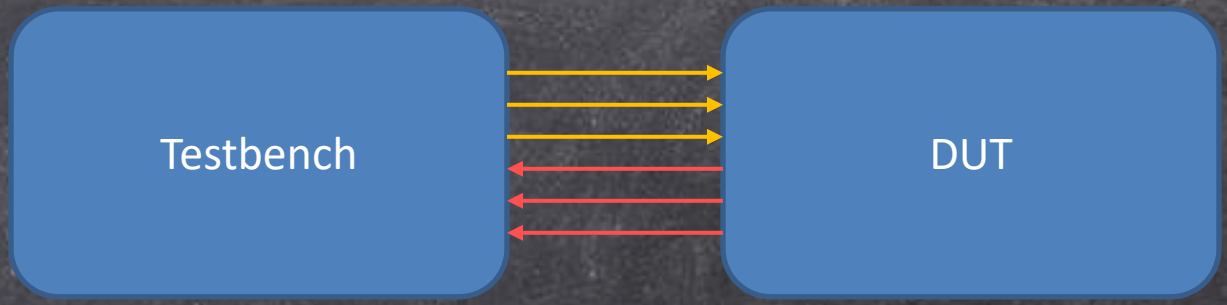
Running example

The DUT


clk
rst
req
addr
0020
wdata
ffff00
ready
done
rw


rst
clk
req
rw
addr[31:0]
wdata[31:0]
ready
done
rdata[31:0]
DUT

Traditional verif environment



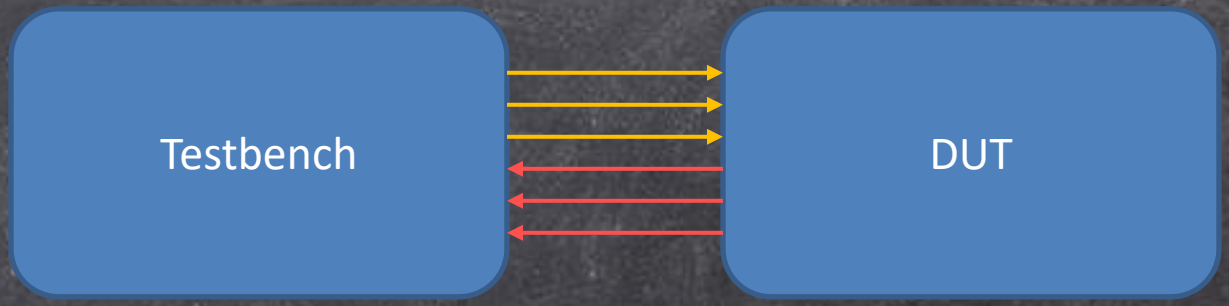

```
module top;
  reg clk, rst
  wire req;
  wire [31:0] addr;
  wire [31:0] data;
  ...
  dut dut_inst(.clk(clk),
               .rst(rst),
               .addr(addr)
               ...);

  testbench tb_inst(.clk(clk),
                    .rst(rst),
                    .addr(addr)
                    ...);
endmodule
```

```
forever begin
  #10 clk = ~clk;
end

initial begin
  rst = 1'b1;
  #100;
  rst = 1'b0;
end
```

Traditional verif environment




```
module testbench;
  initial begin
    req = 1'b0;
    @(posedge rst);
    $display("Starting writing..");
    @(posedge clk);
    req = 1'b1;
    rw = 1'b1;
    addr = 32'h0202_0404;
    wdata = 32'hffff;
    @(posedge done);
    $display("..Done");
  end
endmodule
```

Software inspired features: Data Types

Data types

- ✓Reg, wire, integer, real, time – available in Verilog
- ✓Enums
  - ✓typedef enum {IDLE, READ, WRITE} cmd_t;
- ✓Strings
  - ✓string str = "hey there!";
  - ✓Equality: str1 == str2
  - ✓Concatenation: {str1, str2}
  - ✓str = $sformatf("Value of x is %0d", x);
  - ✓length = str.len();

Data types

- ✓Queues

```
byte myqueue[$];
myqueue.push_back(10);
val = myqueue.pop_front();
```

- ✓Dynamic arrays

```
bit [7:0] dvals[];
dvals = new[dvals_size];
dvals[5] = 23;
```

- ✓Associative arrays

```
int grades[string];
grades["sally"] = 78;
if (grades.exists["tom"]) { do something }
```

Software inspired features:

OOP

OOP

- ✓Encapsulation
  - ✓Class: A type containing properties and methods

```
class Packet;
  cmd_t Command;
  int Done;
  logic [31:0] Data;
  logic [31:0] Addr;

  function int GetStatus();
    return(Done);
  endfunction

  task SetCommand(cmd_t cmd);
    Command = cmd;
  endtask
endclass
```

OOP

- ✓Classes
  - ✓Constructor
  - ✓Objects/instances are dynamically created
  - ✓Variable names holds the object handle
  - ✓No destructor; deallocation happens automatically

```
class Packet;
  ...
  function new();
    Command = IDLE;
    Done = 0;
  endfunction
endclass

Packet myPkt = new();
```

OOP

- Classes
  - Methods and properties can be referenced using the object handle
  - The this keyword is used to refer to the current instance

```
class Packet;
  ...
  function new(cmd_t Command=IDLE, int Done=0);
    this.Command = Command;
    this.Done = Done;
  endfunction
endclass

initial begin
  Packet myPkt = new();
  myPkt.SetCommand(WRITE);
  $display("Status of myPkt is %d", myPkt.Done);
end
```

OOP

- ✓Inheritance
  - ✓Classes can be extended to create derived classes
  - ✓Allows for customization without breaking/rewriting known-good functionality of the base classes

```
class ErrPkt extends Packet;
  bit Error;

  function bit ShowError();
    return(Error);
  endfunction

  task SetCommand(cmd_t cmd);
    Command = ~cmd;
    Error = 1;
  endtask
endclass
```

OOP

```
base = derived;          // Good!
derived = base;          // Won't compile
$cast(derived, base); // Compiles
if ($cast())             // Best
  $fatal(0, "$cast failed, check types");
```

- ✓Polymorphism
  - ✓Define virtual methods and override them
  - ✓Base class handle can call derived class' implementation

```
class Packet;
  logic [31:0] Data;
  virtual function PrintData();
    $display("Packet:: Data = %x", Data);
  endtask
endclass

class ErrPkt extends Packet;
  function new();
    Data = 32'hdead_beef;
  endfunction
  function PrintData();
    $display("ErrPacket:: Data = %x", Data);
  endfunction
endclass
```

```
Packet p;
ErrPacket e;
p = new();
e = new();

p.SetData(32'hg00d);
p.PrintData();
e.PrintData();

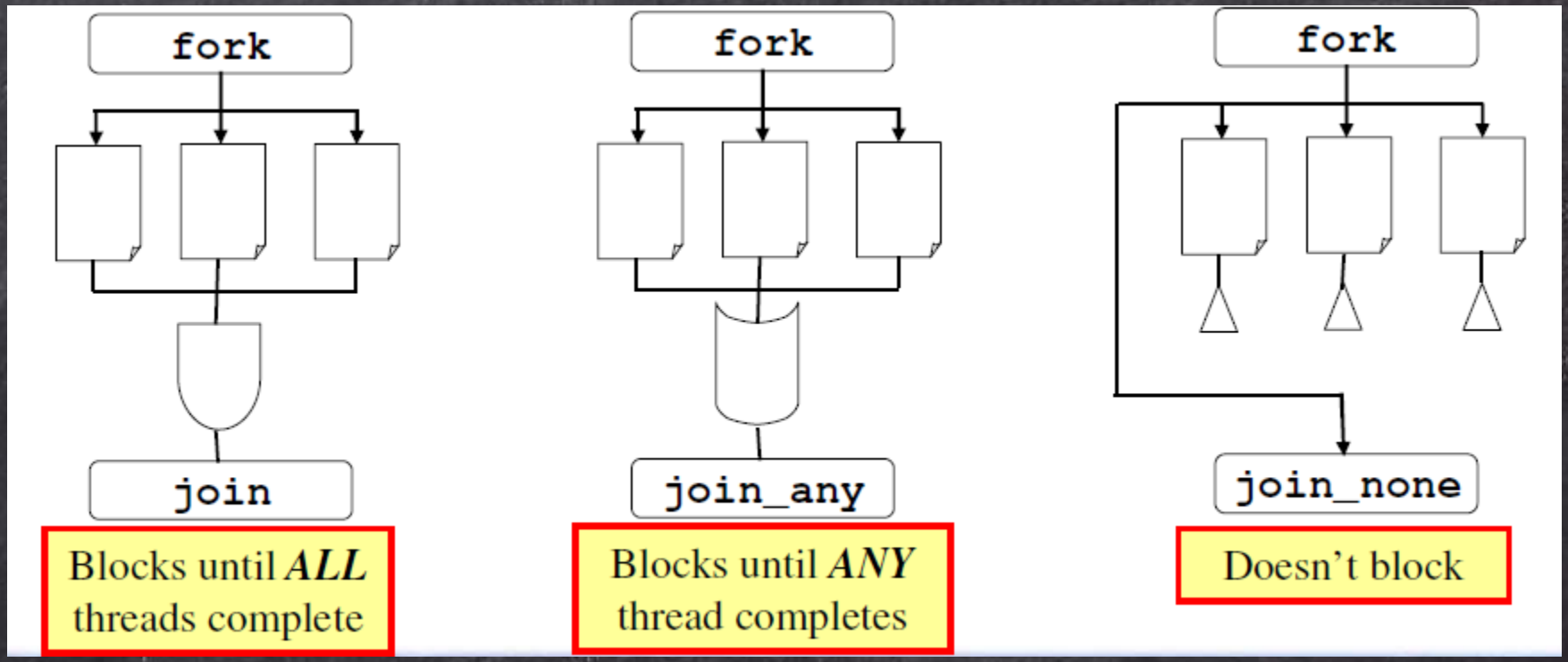
p = e;

p.PrintData();
e.PrintData();
```

Software inspired features: Threads

Threads

- ✓Creating threads
  - ✓fork/join, fork/join_any, fork/join_none



Source: Harry Foster's System Verilog Training slides

Threads

```
task do_test();
fork
  begin
    $display("First Block\n");
    # 20ns;
  end
  begin
    $display("Second Block\n");
    another_task();
  end
join
$display("Done");
endtask

function invoke_test();
  $display("Hello");
  fork
    another_task();
  join_none
  s = a + b;
endfunction
```

```
task get_first(int adr);
  fork
    wait_device(1, adr);
    wait_device(7, adr);
    wait_device(13, adr);
  join_any
  disable fork;
endtask


 task do_test();
   fork
     exec1();
     exec2();
   join_any
   fork
     exec3();
     exec4();
   join_none
   wait fork;
 endtask
```

Threads

- ✓Synchronizing between threads
  - ✓Semaphores
    - ✓Provides control of shares resources
    - ✓A thread needs to procure a *key* before executing
  - ✓Events
    - ✓A thread can *trigger* an *event*
    - ✓Another thread can *wait* on the *event*
  - ✓Mailboxes
    - ✓Share data between threads
    - ✓Behaves like a FIFO + Event

```
event ev;
//Trigger an event
->ev;
//Wait for an event
@ev;
```

Threads

✓Synchronization example using semaphores

```
task read_modify_write(int addr);
  pkt.addr = addr;
  pkt.cmd = READ;
  pkt.send();
  pkt.wait_for_done();
  data = pkt.get_data();

  pkt.data = data+1;
  pkt.cmd = WRITE;
  pkt.send();
  pkt.wait_for_done();

  pkt.cmd = READ;
  pkt.send();
  pkt.wait_for_done();
  new_data = pkt.get_data();

  if(new_data != (data+1)) begin
    $error();
  end
endtask
```

```
semaphore sem;
sem = new(1);

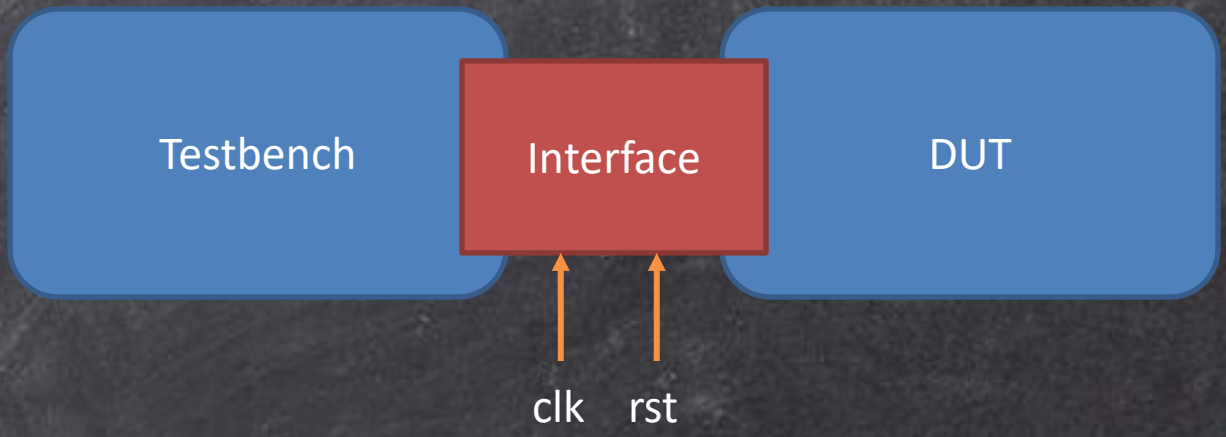
//Thread 1
..do_something..
sem.get(1);
read_modify_write(32'h0000_1111);
sem.put(1);

//Thread2
..do_something..
sem.get(1);
read_modify_write(32'h0000_0110);
sem.put(1);
```

Communication between HW and SW worlds

Connection between HW and SW worlds

- ✓The interface construct
  - ✓An *intelligent* bundle of wires




```
interface dut_if(bit clk, bit rst);
  logic [31:0] Data;
  logic [31:0] Addr;
  logic req;
  logic done;
  logic rw;
  ...
endinterface
```

```
module top;
  bit clk, rst;
  forever #10 clk = ~clk;

  dut_if dut_if_inst(clk, rst);
  dut dut_inst(dut_if_inst);
  testbench tb_inst(dut_if_inst);
endmodule
```

Connection between HW and SW worlds

- ✓Virtual Interface
  - ✓A handle/pointer to a physical interface
  - ✓Used on the SW side to access the HW side
  - ✓You can control/observe wires of the DUT using a vif

```
class VifExample;
  virtual dut_if vif;

  function new(dut_if vif);
    this.vif = vif;
  endfunction

  extern task ControlAndObserve();
endclass
```

```
task VifExample::ControlAndObserve();
  $display("Starting writing..");
  @(posedge vif.clk);
  vif.req = 1;
  vif.wr_en = 1;
  vif.Addr = 32'h0202_0101;
  vif.Data = 32'hfff;
  @(posedge vif.done);
  $display("..Done");
endtask
```

Connection between HW and SW worlds

- ✓Virtual Interface
  - ✓A handle/pointer to a physical interface
  - ✓Used on the SW side to access the HW side
  - ✓You can control/observe wires of the DUT using a vif

```
module top;
  bit clk, rst;
  forever #10 clk = ~clk;

  dut_if dut_if_inst(clk, rst);
  dut dut_inst(dut_if_inst);
  testbench tb_inst(dut_if_inst);

  initial begin
    VifExample ve = new(dut_if_inst);
    ve.ControlAndObserve();
  end
endmodule
```

Hardware inspired features: Stimulus

Stimulus

- Applying inputs to stimulate the DUT
- Types
  - Active stimulus
    - Drive in various command packets into the DUT
  - Passive stimulus
    - Respond to something that happened in the DUT, e.g. an error condition or an interrupt

Stimulus

- What can you vary on the inputs?
  - Values (eg: Different values of Data, Addr, etc)
  - Timings and order (eg: Pkt1 -> delay -> Pkt2 -> etc)
  - How many interfaces to wiggle?
  - Etc...
- General philosophy
  - Represent various meaningful operations in tasks
  - Fork off multiple tasks to have collisions
- General intent
  - Exercise as many scenarios as possible

Stimulus

- ✓Randomization
  - ✓$random
    - ✓Addr = $random
  - ✓$urandom
    - ✓Returns a random unsigned number
  - ✓$urandom_range
    - ✓Returns a random_unsigned number in the given range
    - ✓X = $urandom_range(200,100)
  - ✓randomize()
    - ✓randomize(x, y, z)

Stimulus

- ✓Randomization inside classes
  - ✓Declaration of a random variable
  - ✓Writing a constraint
  - ✓Calling .randomize()

```
class Packet;
  rand bit [31:0] Addr;

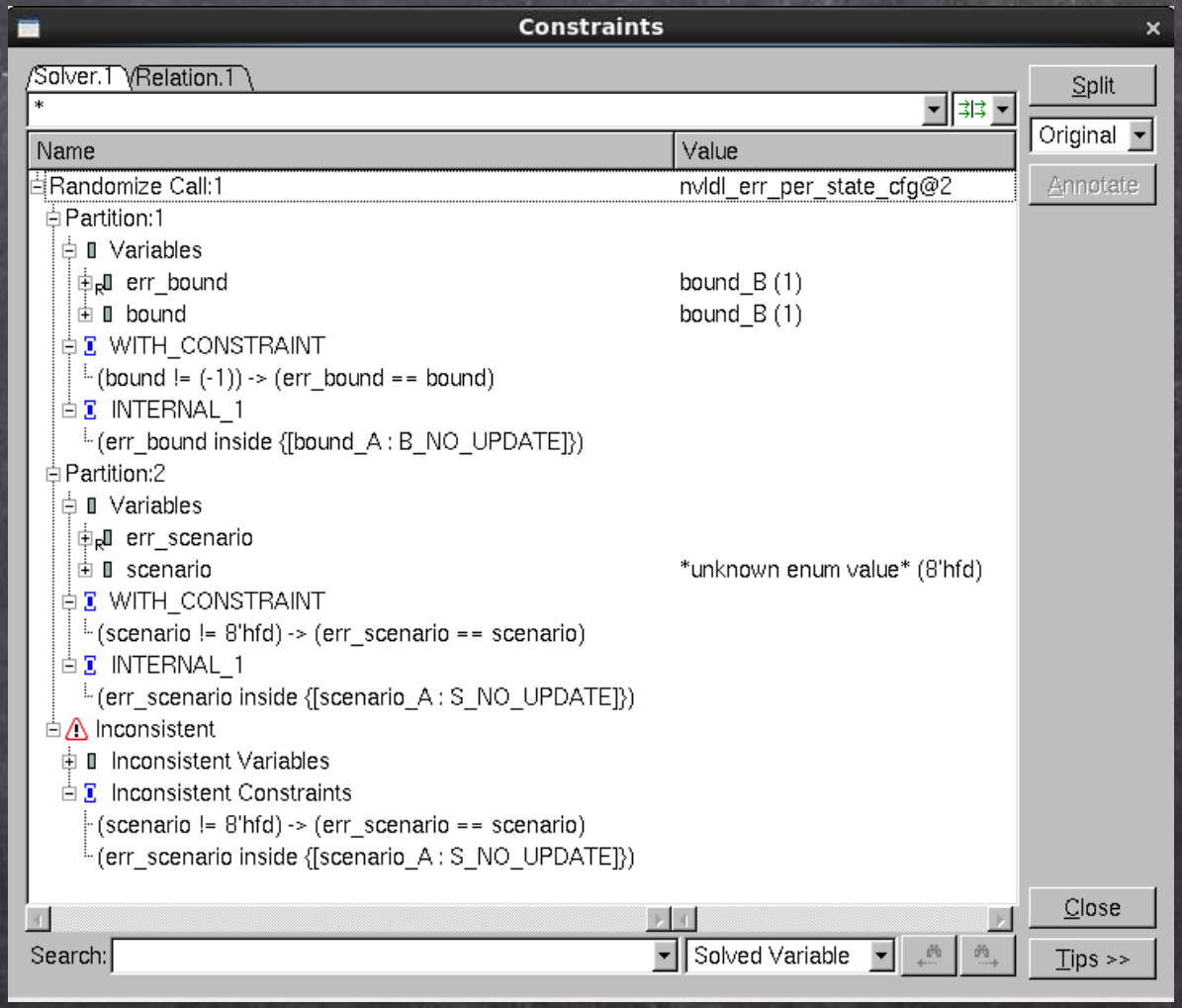
  constraint addr_c {
    Addr >= 32'h0000_ffff;
    Addr <= 32'hffff_0000;
    Addr[1:0] == 2'b00;
  }
endclass
```

```
initial begin
  Packet pkt = new();
  if (pkt.randomize()) begin
    $display("Addr is %h", pkt.Addr);
  end else begin
    $display("Randomization failed");
  end
  pkt.send();
end
```

Stimulus

- ✓ Constraints
  - ✓ Set membership
    - ✓ Addr inside {0, 100, [1000:2000]};
  - ✓ Distribution
    - ✓ Addr dist { 0 := 50, [100:200] := 20, 250 := 30 };
  - ✓ Implication
    - ✓ if (Command==WRITE) Addr > 32'h8000;
    - ✓ else Addr == 32'hffff_0000;

Aside: Constraint Solver



Hardware inspired features: Checking

Checking

- Ensuring that the DUT is behaving correctly
- Checkers are the parts of the testbench that throw errors when something illegal happens
  - Signal level checking
    - Eg. Assertions
  - Behavioral checking
    - Eg. Scoreboards or end-to-end checkers ("If X went in, Y should be the output")
    - Eg. State machine checkers ("FSM should not go into state X, unless Y happens")
    - Eg. Register checkers ("Register X should have a value Y, if Z happens")

Checking

- Assertions
  - An assertion is a declaration of design intent
  - System Verilog has lots of constructs for writing assertions
  - Written by:
    - Design engineers along with RTL
    - Verification engineers in interfaces
  - Example:
    - If req rises, then ready should be asserted within 2-20 clock cycles

```
assert property (@(posedge Clk) $rose(Req) |-> ##[2:20] Ready);
```

Checking

- ✓ Assertions
  - ✓ Temporal delays
    - ✓ Req ##[5:10] Done
  - ✓ Repetition
    - ✓ Start[*2:5]
  - ✓ Implication
    - ✓ Req |-> (A && B);
  - ✓ System functions
    - ✓ $rose(sig)
    - ✓ $fell(sig)
    - ✓ $onehot(sig)

Checking

- ✓Assertions

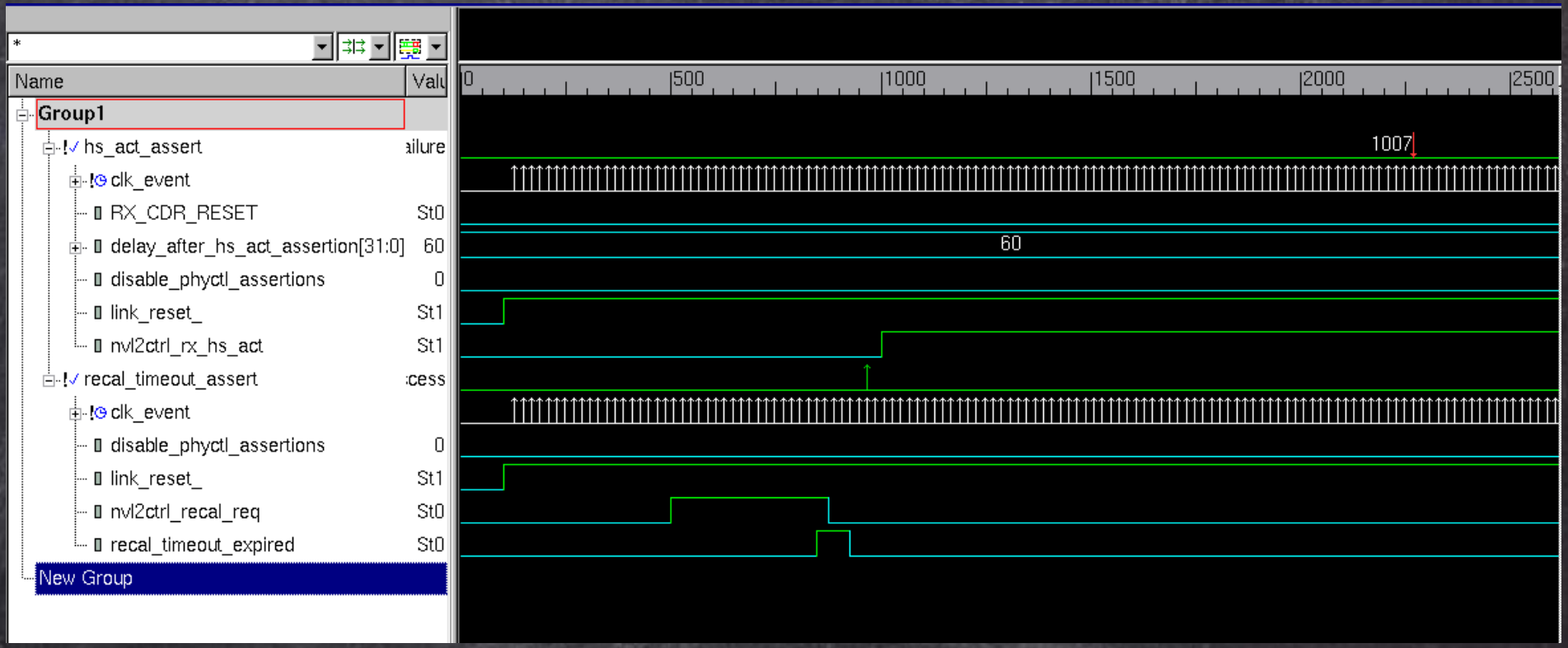
```
property match_value;
  byte temp;
  @(posedge clk) disable iff (!rst)
  ($rose(req), temp=data) |-> ##[10:20] ((ready==1) && (wdata==temp));
endproperty
assert property (match_value);

property myprop;
  @(sig1, sig2) disable iff (x > y)
  sig1 |=> $fell(sig2) ##1 !sig1;
endproperty
assert property myprop();

property onehot_and_valid(mysig);
  @(posedge clk)
  ((|mysig != 1'bx) && ($countones(mysig)==1))
endproperty
assert property onehot_and_valid(data);
```

Checking

- ✓Assertions



Hardware inspired features: Coverage

Coverage

- Quantitative measure of how well the stimulus exercised the DUT
  - Run all test, merge coverage results and generate a report
- Ensuring that the DUT was exposed to all possible (read, *interesting*) scenarios
- Types of coverage
  - Code coverage
  - Functional coverage

Coverage

- ✓Functional coverage
  - ✓Need to write code to express what you intend to cover
  - ✓Two ways:
    - ✓ Cover properties
    - ✓ Covergroups

Coverage

- ✓ Functional coverage
  - ✓ Covergroups
    - ✓ Used for expressing combinations and for expressing coverage at a transaction level
    - ✓ Can be written in modules or interfaces or classes

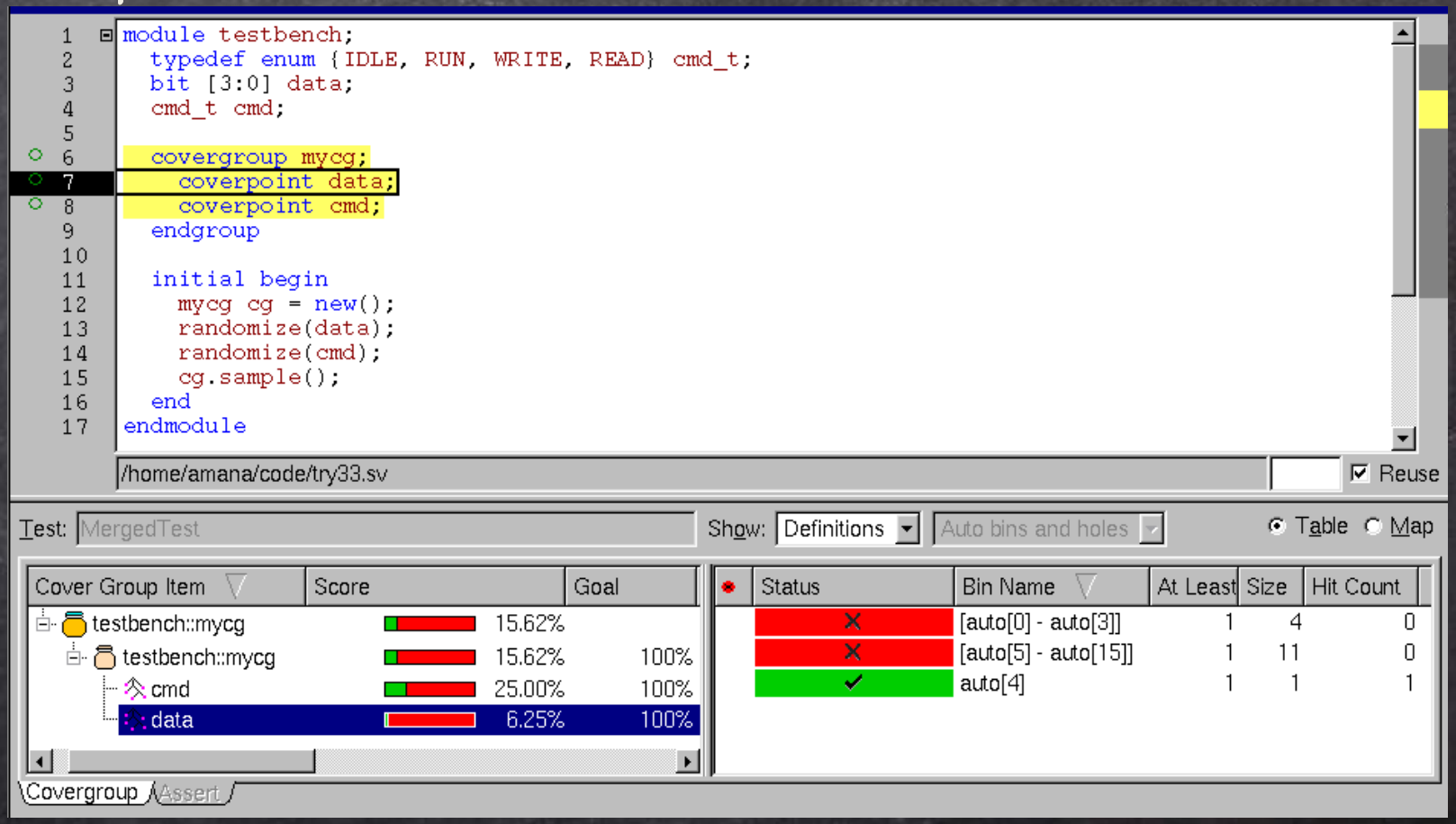
```
module testbench;
  bit [3:0] data;
  cmd_t cmd;

  covergroup mycg;
    coverpoint data;
    coverpoint cmd;
  endgroup

  initial begin
    mycg cg = new();
    randomize(data);
    randomize(cmd);
    cg.sample();
  end
endmodule
```

Coverage

- ✓Functional coverage
  - ✓Report



Coverage

- ✓Functional coverage
  - ✓Defining bins
  - ✓Cross coverage

```
bit [7:0] addr;
cmd_t cmd;

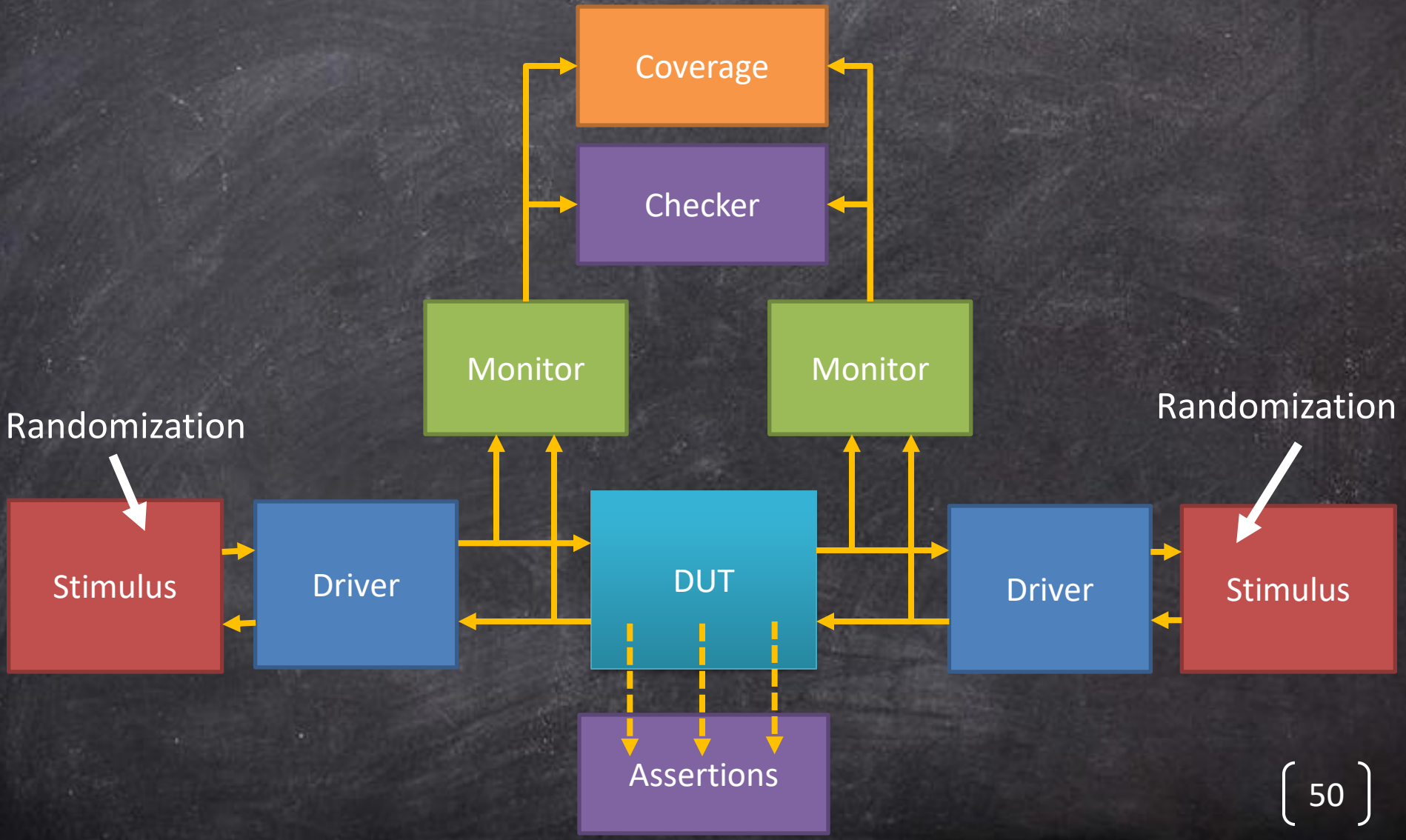
covergroup mycg;
  addr_cp: coverpoint addr {
    bins low  = {[1   : 63]};
    bins med  = {[64  : 127]};
    bins high = {128 : 254]};
    ignore_bins ig = 255;
    illegal_bins zero = 0;
  };
  cmd_cp: coverpoint cmd;
  data_cmd_cr: cross addr_cp, cmd_cp;
endgroup
```

Coverage

- ✓ Functional coverage
  - ✓ Covergroups in classes
    - ✓ Can cover any class member
    - ✓ Can cover any signals using an interface handle
    - ✓ Create in the class constructor
    - ✓ Sample in any task/function (e.g. when packet is available)

Putting it all together – A Testbench

A Testbench


Coverage
Checker
Monitor
Monitor
Randomization
Randomization
Stimulus
Driver
DUT
Driver
Stimulus
Assertions

Aside: DPI

- ✓Direct Programming Interface
- ✓Can invoke C functions from within System Verilog code, and vice versa
- ✓Verilog had PLI. System Verilog added DPI.
  - ✓PLI 1.0 – tf routines
  - ✓PLI 2.0 – acc routines
  - ✓PLI 3.0 – vpi routines

- TF library (introduced in 1985)
  - Provides access to system task/function arguments
  - Provides synchronization to simulation time and events
- ACC library (introduced in 1989)
  - Extends TF library to access to design structure
  - Created to enable ASIC delay and power calculation
- VPI library (introduced in 1995)
  - Superset of TF and ACC libraries
  - Adds access to behavioral and RTL models

- ✓Very powerful for multi-language verification environments. Eg. checker written in C/C++.
- ✓A nice paper on DPI

Aside: DPI

C invoked from SV

```
module top;
  import "DPI" function int cfunc();
  initial begin
    ...
    x = cfunc();
    ...
  end
endmodule
```

```
#include <stdio.h>
int cfunc() {
  printf("Hello from cfunc!");
  return 0;
}
```

SV invoked from C

```
module top;
  export "DPI" function vfunc(int a, int b);
  function void vfunc(int a, int b) {
    $display("Hello from vfunc!");
  }
endmodule
```

```
#include <stdio.h>
extern void vfunc(int, int);
int any_func() {
  ...
  vfunc();
  ...
  return 0;
}
```

Final Thoughts

Final thoughts

- ✓System Verilog adds a whole lot more on top of Verilog
  - ✓Most new features used by verification engineers
- ✓It is the most common verification language in the industry (the other one is C++, I think)
- ✓Verification is a very dynamic and challenging field
- ✓Knowing System Verilog will help you in your job search ☺

Synopsys Training

- ✓Please complete this training: SystemVerilog for Verification

Thanks!



That’s a lot of information

Backup

Aside: Final block

- Initial blocks execute at the beginning of the simulation
  - Order is not guaranteed
- Final block execute at the end of the simulation
  - Order is not guaranteed
  - You are now allowed to consume time in a final block

Threads

✓Mailboxes

```
mailbox #(packet_t) mbox;
mbox = new(10);
//Sender thread
mbox.put(pkt);
//Receiver thread
mbox.get(pkt);
```

new()	Create a mailbox with a specified number of slots
num()	Return number of items in mailbox
get()	Retrieve an item from the mailbox, if empty block until an item is available
try_get()	Retrieve an item from the mailbox, if empty do not block
peek()	Copy an item from the mailbox, if empty block until an item is available
try_peek()	Copy a item from the mailbox, if empty do not block
put()	Put an item in the mailbox, if full block until an space is available
try_put()	Put an item in the mailbox, if full do not block

Source: Harry Foster's System Verilog Training slides

OOP

- ✓Inheritance
  - ✓local
  - ✓protected
  - ✓super

```
class Sub extends Base;
  int a;
  function new();
    // prepend …
    super.Put();…
    // append …
  endfunction

  task Set_a(input int c);
    a = c;
    super.a = c+1;
    Set_i(c);
  endtask
endclass
```

```
class Base;
  local int i;
  int a,d;
  protected task Set_i(input int i);
    this.i = i;
  endtask
  function Put();
    ...
  endfunction
endclass


initial begin
  Sub S = new();
  S.i = 4;     // illegal i local to Base
  S.Set_i(4); // illegal Set_i protected
  S.a = 5;     // legal Base::a hidden by
               // Sub::a
  S.set_a(5); // legal Set_a unprotected
  S.d = 3;     // legal
end
```

OOP

- ✓Polymorphism
  - ✓Abstract classes
    - ✓Used to define API. Can not be instantiated

```
virtual class BasePacket;
  int id;
  pure virtual task send(bit[63:0] data);
  pure virtual task receive(bit[63:0] data);
  pure virtual task printme();
endclass

class ATMPacket extends BasePacket;
  bit header[3:0];
  task send (bit[63:0] data);
    // implementation described here
    ...
  endtask
endclass
```

Stimulus

- ✓ Constrained randomization
  - ✓ Inline constraints
    - ✓ `Pkt.randomize() with { Addr == 32'b0000_0000;};`
  - ✓ Ordering
    - ✓ `Constraint order_c { solve Addr before Data; }`
  - ✓ Enable/disable random variables
    - ✓ `Addr.rand_mode(0);`
  - ✓ Enable/disable constraints
    - ✓ `Addr_c.constraint_mode(0);`

Checking

- ✓Assertions
  - ✓Sequences
  - ✓Properties

```
sequence cmd_req(cmd);
  ($rose(req) && (rw==cmd));
endsequence

sequence data_read;
  ##[2:20] (ready==1) ##[50:100] (rdata!='x);
endsequence

sequence data_write;
  ##[2:20] (ready==1) ##0 (wdata!='x);
endsequence

property read_prop;
  @(posedge clk) cmd_req(0) |-> data_read |-> done;
endproperty

property write_prop;
  @(posedge clk) cmd_req(1) |-> data_write |-> done;
endproperty

read_assert:  assert property (read_prop);
write_assert: assert property (write_prop);
```

Checking

- ✓Behavioral checker example

```
class mychecker;
  virtual dut_in_if ivif;
  virtual dut_out_if ovif;
  task main();
    forever begin
      begin
        reset_checker();
        @(negedge ivif.rst);
      end
      fork
        detect_reset();
        track_inputs();
        track_outputs();
        do_comparisons();
      join_any
      disable fork;
    end
  endtask
endclass
```

```
task track_inputs();
  forever begin
    @(posedge ivif.done);
    create_inp_pkt(pkt);
    in_queue.push_back(pkt);
  end
endtask

task track_outputs();
  forever begin
    @(posedge ovif.done);
    create_out_pkt(pkt);
    out_queue.push_back(pkt);
  end
endtask
```

Checking

- Behavioral checker example

```
task do_comparisons();
  forever begin
    if ((in_queue.size()>0) && (out_queue.size()>0)) begin
      inp = in_queque.pop_front();
      oup = out_queue.pop_front();
      if(!is_match(inp,oup))
        $error("Mismatching packets found");
    end
endtask


module top;
  mychecker checker;
  initial begin
    checker = new();
    checker.vif = dut_if_inst;
    checker.main();
  end
endmodule
```

Coverage

- Code coverage (not new in System Verilog)
  - Types
    - Line coverage
    - Toggle coverage
    - FSM coverage
    - Branch coverage
  - No coding required
  - Just instrument the code by passing switches to the compiler
  - Simulator produces a report

Coverage

- ✓Functional coverage
  - ✓Cover properties
    - ✓Used for expressing coverage at signal level
    - ✓Use the same assertion syntax
    - ✓Instead of "asserting" a property, "cover" a property
    - ✓Generally written in interfaces

```
// done asserting few cycles before fom_req de-asserting
property expiry_case;
    @(posedge clk) disable iff (!reset)
    $rose(done) |-> ##[1:5] ($fell(req) && (done==1'b0));
endproperty
expiry_case_cprop: cover property (expiry_case);
```

Coverage

```
class PacketCov extends Packet;

  covergroup mycg1;
    addr_cp: coverpoint Addr;
    cmd_cp: coverpoint Cmd;
    data_cmd_cr: cross addr_cp, cmd_cp;
  endgroup

  covergroup mycg2;
    sig_cp: coverpoint (vif.sig1 && vif.sig2);
  endgroup

  function new();
    mycg1 = new();
    mycg2 = new();
  endfunction

  function sample_cgs();
    mycg1.sample();
    mycg2.sample();
  endfunction

endclass
```

```
class monitor;
  PacketCov pktcov;
  ...
  task monitor_signals();
    forever begin
      @(posedge vif.done);
      pktcov.sample_cgs();
    end
  endtask
endclass
```

Aside: Modelling and Abstraction

- A model is an artifact which represents a physical entity and is used to perform some analysis on it.
- A model 'abstracts' a physical system.
- A model includes only the details required for analysis
  - A behavioral RTL model of a counter captures only the functionality.
  - A black-box model of an IP contains only the interface details.
  - A transistor level model of an ALU captures timing as well as power consumption information.
- Models are not good if they can't be executed (simulated). Some models can be implemented (synthesized).

Aside: Modelling and Abstraction

- As we go down the design flow (i.e. as we move from specification to implementation), the level of abstraction decreases
  - Algorithm -> RTL -> Gates -> Transistors
- Models at lower levels of abstraction have more details and hence, take too much time to execute/analyze/simulate
  - Full chip transistor level simulation would take several years, and the chip won't reach the customers in time
- To make the task of verification easier, we need to come up with simpler, yet powerful, abstractions
  - That is why System Verilog introduced programming language like features. These features enable us to code powerful models at higher levels of abstraction.